

实验六报告

唐宇恩 251220100

2026 年 6 月 10 日

1 控制器设计实验

端口定义：

端口名称	端口功能
opcode	指令操作码
funct3	指令功能码
funct7	指令功能码
ExtOp	立即数扩展类型
RegWr	寄存器写使能
BandJ	分支和跳转指令标志
ALUctr	ALU 控制信号
ALUAsrc	ALU 输入 A 选择信号
ALUBsrc	ALU 输入 B 选择信号
MemWr	数据存储器写使能
MemtoReg	寄存器写数据来源选择信号
MemWr	数据存储器写使能

$$\begin{aligned}
\text{RegWr} &= \text{lui} + \text{auipc} + \text{jal} + \text{jalr} + \text{R} + \text{I-Arithmetic} + \text{load} \\
\text{ALUASrc} &= \text{auipc} + \text{jal} + \text{jalr} \\
\text{ALUBSrc}[0] &= \text{jal} + \text{jalr} \\
\text{ALUBSrc}[1] &= \text{lui} + \text{auipc} + \text{I-Arithmetic} + \text{load} + \text{store} \\
\text{MemWr} &= \text{store} \\
\text{MemtoReg} &= \text{load} \\
\text{BandJ} &= \text{Branch}(\text{Branch} * 4 + \text{funct3}[2] * 2 + \text{funct3}[0]) + \text{jalr} * 2 + \text{jal} * 1 \\
\text{ExtOp} &= (\text{lui} + \text{auipc}) * 1 + (\text{I-Arithmetic} + \text{load} + \text{jalr}) * 0 + \text{jal} * 4 + \text{store} * 2 + \text{Branch} * 3 \\
\text{ALUctr} &= 0 * (\text{auipc} + \text{jal} + \text{jalr} + \text{load} + \text{store}) + 15 * \text{lui} + (14 + \text{fuct3}[1]) * \text{Branch} \\
&\quad + (\text{funct3} + \text{funct7}[5]) * \text{R} + (\text{funct3} + 8) * \text{I-Arithmetic}
\end{aligned}$$

1.1 实验原理图和电路图

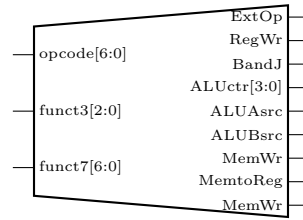


图 1: 控制器原理图

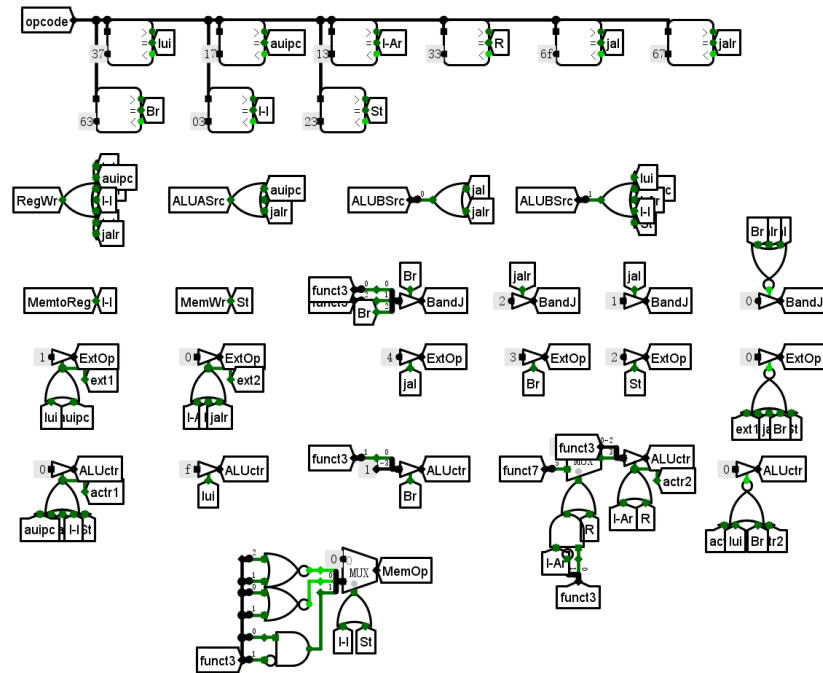
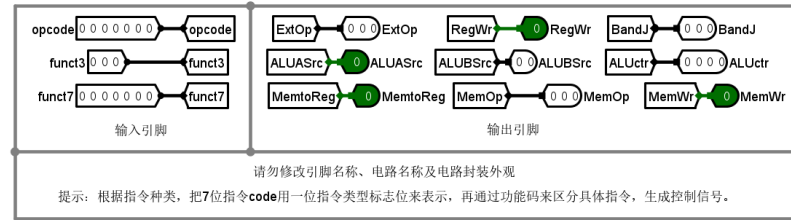
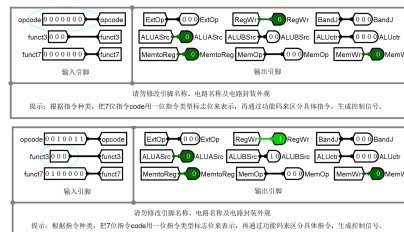
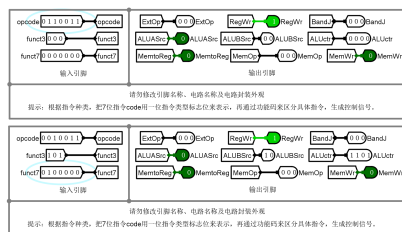


图 2: 控制器电路图

1.2 仿真测试



1.3 错误分析

使用三态门遇到未定义的情况会输出高阻态，实际测试要求输出 0；需要使用或门将所有情况覆盖，保证输出为 0 或 1。

2 单周期 CPU 设计实验

2.1 整体方案设计

端口定义：

端口名称	端口功能
clk	时钟信号
ReSet	复位信号
Init Addr	指令存储器初始地址
PC	程序计数器
IR	指令寄存器
busW	寄存器写数据总线
Din	数据存储器输入数据总线
cycle	时钟周期计数器
halt	停机信号

2.2 实验原理图和电路图

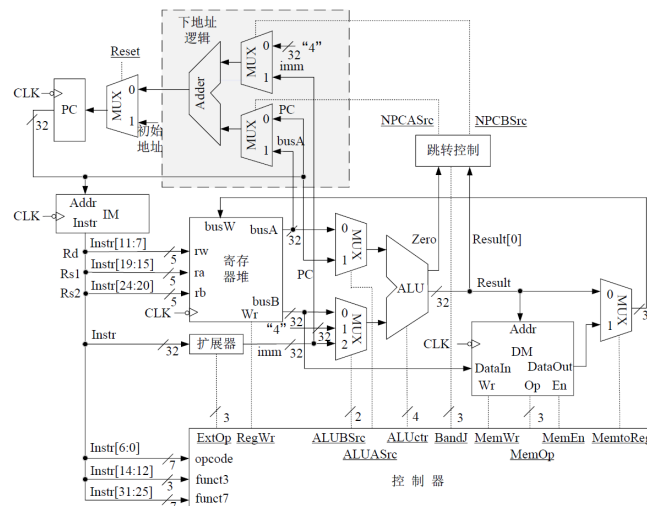


图 4: 单周期 CPU 原理图

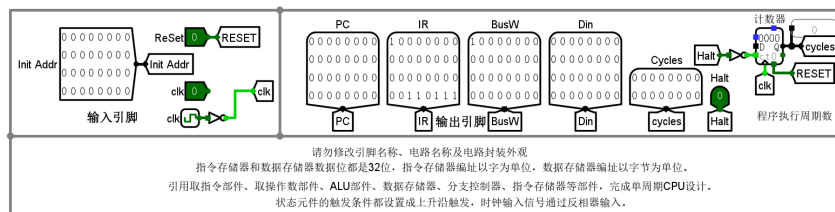


图 5: 单周期 CPU 电路图

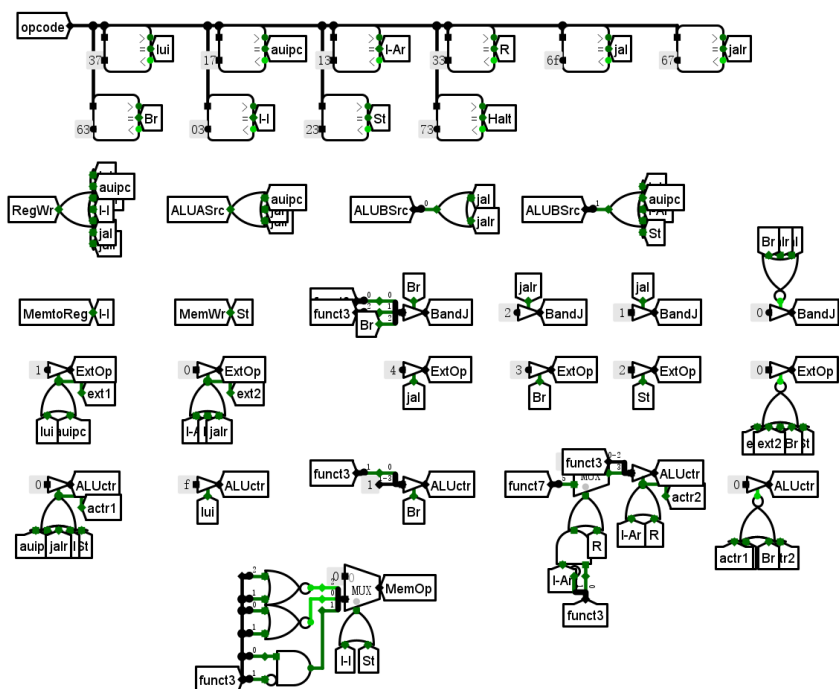
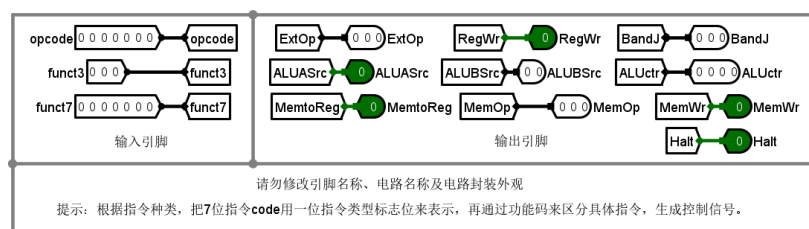


图 6: 控制器电路图

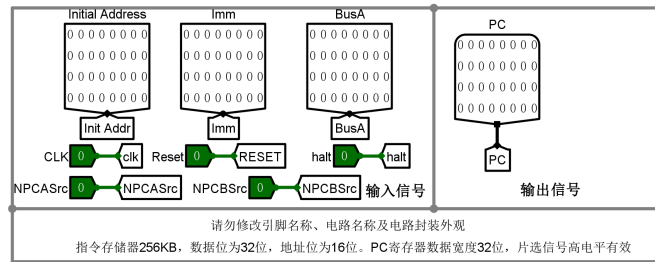


图 7: 指令获取单元电路图

2.3 仿真测试

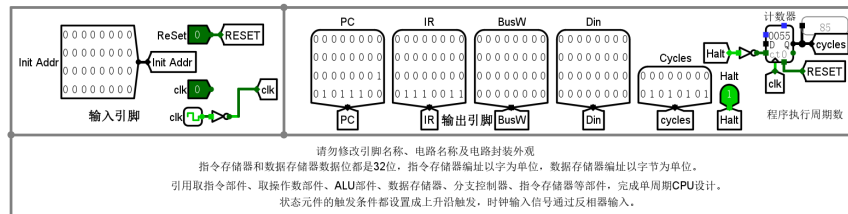


图 8: 单周期 CPU 仿真测试

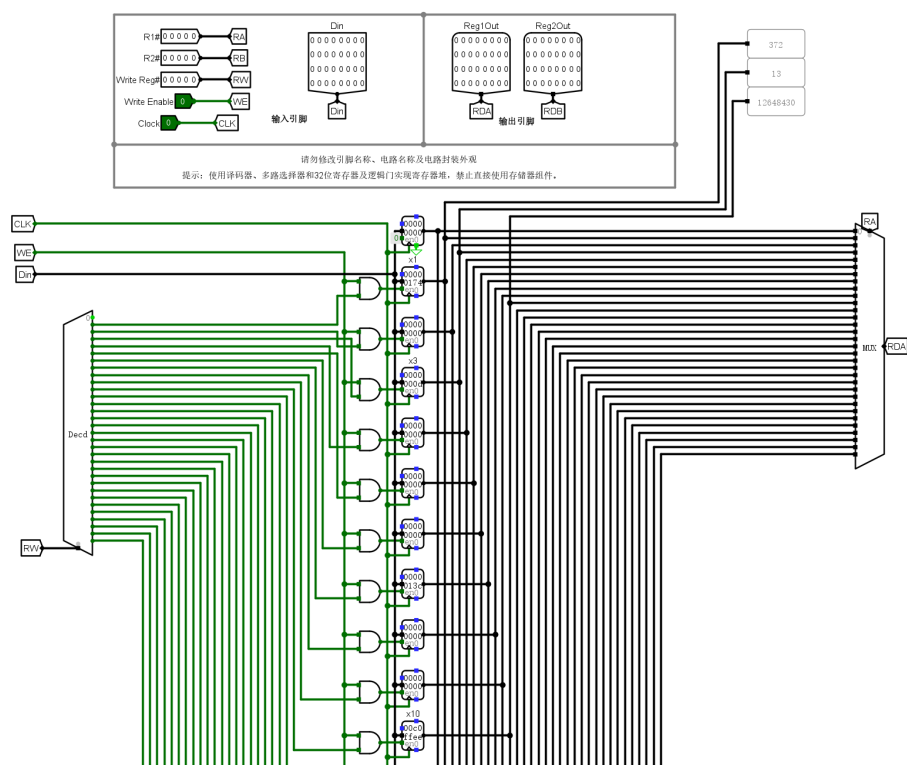


图 9: 寄存器组值

2.4 错误分析

一开始出现如下报错：

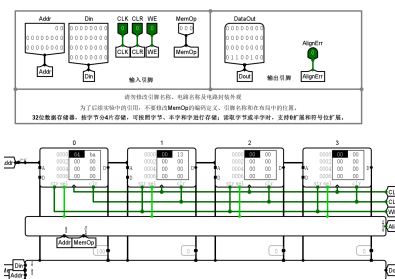
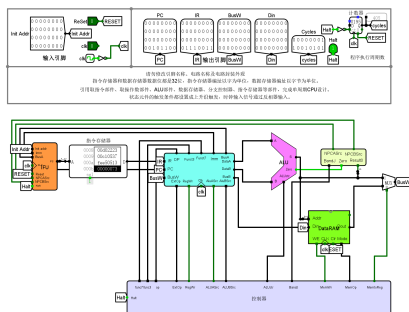
00000044	0f812193	00f000f0	00000000	0011	00000044	0f812193	00f000f0	00000000	0011
00000048	00100193	00000000	00f000f0	0012	00000048	00100193	00000000	00f000f0	0012
0000004c	10779ac3	00000000	00f000f0	0013	0000004c	10779ac3	00000000	00f000f0	0013
00000050	00100093	00000001	0f000f0f	0014	00000050	00100093	00000001	0f000f0f	0014
00000054	01f00113	00000000	00000000	0015	00000054	01f00113	00000000	00000000	0015
00000058	00209813	00000000	0000001f	0016	00000058	00209813	00000000	0000001f	0016
0000005c	80000057	00000000	00000000	0017	0000005c	80000057	00000000	00000000	0017
00000060	00402123	00000001	00000000	0018	00000060	00402123	00000001	00000000	0018
00000064	0c7010c3	00000000	00000000	0019	00000064	0c7010c3	00000000	00000000	0019
00000068	80000057	00000000	00000000	001a	00000068	80000057	00000000	00000000	001a
0000006c	00c01113	00000000	7fff0000	001b	0000006c	00c01113	00000000	7fff0000	001b
00000070	402000b3	fff00000	00000000	001c	00000070	402000b3	fff00000	00000000	001c
00000074	fff00057	fff00000	00000000	001d	00000074	fff00057	fff00000	00000000	001d
00000078	00c01113	00000005	00000000	001e	00000078	00c01113	00000005	00000000	001e
0000007c	0c701263	00000000	fff00000	001f	0000007c	0c701263	00000000	fff00000	001f
00000080	80000057	00000000	00000000	0020	00000080	80000057	00000000	00000000	0020
00000084	7ff00013	00000001	00000000	0021	00000084	7ff00013	00000001	00000000	0021
00000088	00100193	00000001	00000000	0022	00000088	00100193	00000001	00000000	0022
0000008c	00400193	00000000	00000000	0023	0000008c	00400193	00000000	00000000	0023
00000090	0c791063	00000000	00000001	0024	00000090	0c791063	00000000	00000001	0024
00000094	80000057	00000000	00000000	0025	00000094	80000057	00000000	00000000	0025
00000098	7ff00013	00000000	00000000	0026	00000098	7ff00013	00000000	00000000	0026
0000009c	00001193	00000000	00000000	0027	0000009c	00001193	00000000	00000000	0027
000000a0	0c700193	00000007	00000000	0028	000000a0	0c700193	00000007	00000000	0028
000000a4	0c7990c3	00000000	00000000	0029	000000a4	0c7990c3	00000000	00000000	0029
000000a8	00000057	00000005	00000000	002a	000000a8	00000057	00000005	00000000	002a
000000ac	0c800093	00000174	00000000	002b	000000ac	0c800093	00000174	00000000	002b
000000b0	00400113	00000000	7fff0000	002c	000000b0	00400113	00000000	7fff0000	002c
000000b4	00c00023	00000174	00000000	002d	000000b4	00c00023	00000174	00000000	002d
000000b8	0f000113	000000ff	00000000	002e	000000b8	0f000113	000000ff	00000000	002e
000000bc	00200063	00000175	000000ff	002f	000000bc	00200063	00000175	000000ff	002f
000000c0	0c000113	00000000	00000000	0030	000000c0	0c000113	00000000	00000000	0030
000000c4	00c00123	00000176	00000000	0031	000000c4	00c00123	00000176	00000000	0031
000000c8	00000113	00000000	00000000	0032	000000c8	00000113	00000000	00000000	0032
000000cc	00c001a3	00000177	00000000	0033	000000cc	00c001a3	00000177	00000000	0033
000000d0	00100003	ffffff00	00000174	0034	000000d0	00100003	ffffff00	00000174	0034
000000d4	fff00013	ffffff00	00000000	0035	000000d4	fff00013	ffffff00	00000000	0035
000000d8	00000193	00000000	00000000	0036	000000d8	00000193	00000000	00000000	0036
000000dc	007010c3	00000000	ffffff00	0037	000000dc	007010c3	00000000	ffffff00	0037
000000e0	00000063	0000ff00	00000000	0038	000000e0	00000063	0000ff00	00000000	0038
000000e4	00010307	00010000	00000000	0039	000000e4	00010307	00010000	00000000	0039
000000e8	fff00057	0000ff00	7fff0000	003a	000000e8	fff00057	0000ff00	7fff0000	003a
000000ec	00000193	00000000	00000000	003b	000000ec	00000193	00000000	00000000	003b
000000f0	0c701063	00000000	0000ff00	003c	000000f0	0c701063	00000000	0000ff00	003c
000000f4	01000123	00000178	0000ff00	003d	000000f4	01000123	00000178	0000ff00	003d
000000f8	00c01963	00000000	00000000	003e	000000f8	00c01963	00000000	00000000	003e
000000fc	0c000193	00000000	00000000	003f	000000fc	0c000193	00000000	00000000	003f
00000100	00000193	00000000	00000000	0040	00000100	00000193	00000000	00000000	0040
00000104	04701063	00000000	00000000	0041	00000104	04701063	00000000	00000000	0041
00000108	01000123	00000178	00000000	0042	00000108	01000123	00000178	00000000	0042
0000010c	00010063	00c0ff00	00000000	0043	0000010c	00010063	00c0ff00	00000000	0043
00000110	00c10000	00000000	00000000	0044	00000110	00c10000	00000000	00000000	0044
00000114	fff00057	00c0ff00	7fff0000	0045	00000114	fff00057	00c0ff00	7fff0000	0045
00000118	00400193	00000000	00000000	0046	00000118	00400193	00000000	00000000	0046
0000011c	04701263	00000000	00c0ff00	0047	0000011c	04701263	00000000	00c0ff00	0047
00000120	00000063	00c0ff00	00000000	0048	00000120	00000063	00c0ff00	00000000	0048
00000124	00c10307	00c10000	00000000	0049	00000124	00c10307	00c10000	00000000	0049

图 10: 测试错误

经过对指令的反汇编，发现是 x3 和 x4 寄存器的值有错误存取的情况，经过仔细检查，是寄存器堆中 x3 和 x4 的输出线短接。修复后测试通过。

2.5 累加程序测试实验

2.6 测试截图

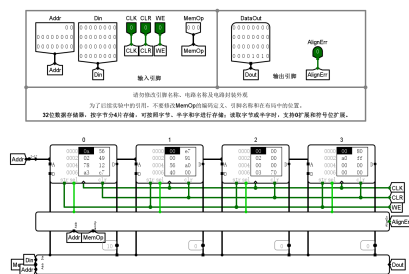
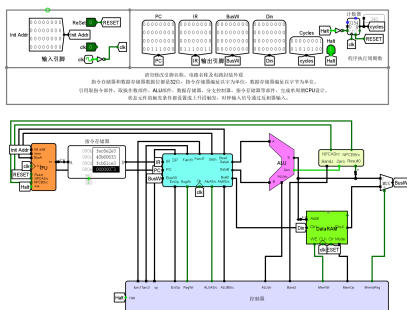


2.7 错误分析

本次实验没有遇到错误。

3 累加程序测试实验

3.1 测试截图

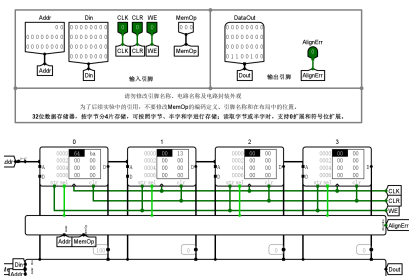
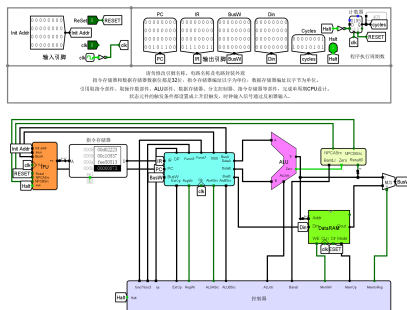


3.2 错误分析

本次实验没有遇到错误。

4 冒泡排序程序测试实验

4.1 测试截图



4.2 错误分析

本次实验没有遇到错误。

5 官方测试集实验

5.1 测试电路图设计

为了能够在 test 电路中直接输出寄存器的值，需要对 Regfile, IDU 和 CPU 进行修改，增加三个新的端口 x10, x3, x1，以输出所需寄存器的值。同时增加一个新的 test 电路，在左侧电路图浏览器中右键“设置为主电路”，并将 x10 的值与 0x00c0ffee 进行比较，输出 correct 信号。同时，需要注意在 CPU 电路中增加一个新的 halt 计算电路。

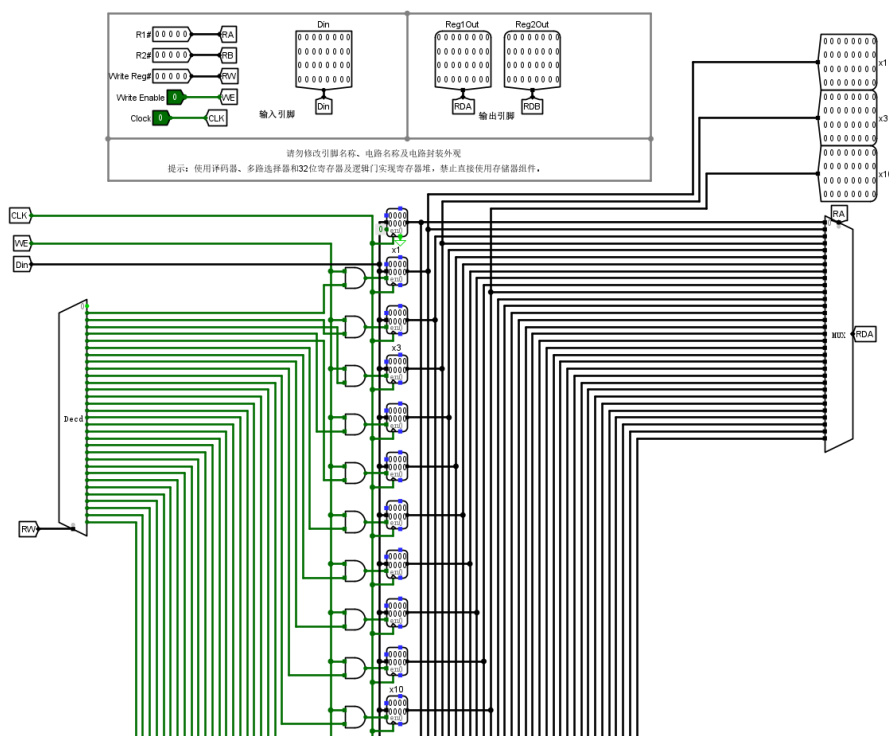


图 14: 修改后的寄存器文件电路图

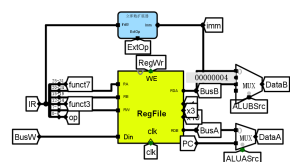
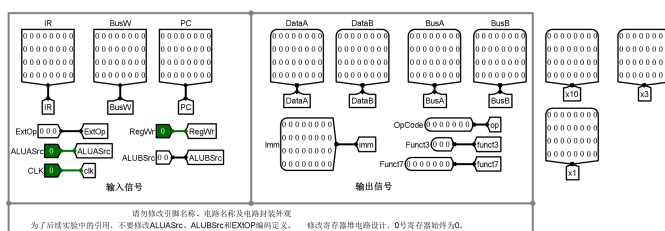


图 15: 修改后的 IDU 电路图

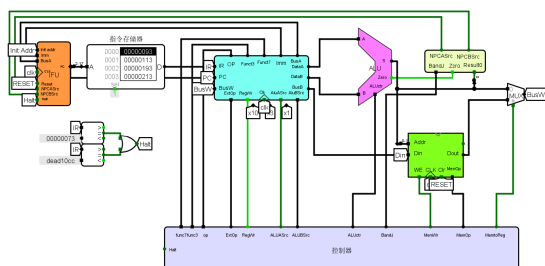
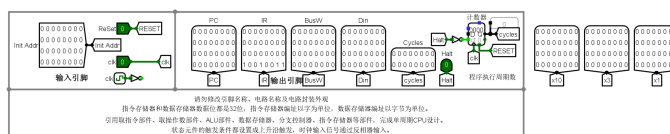


图 16: 修改后的 CPU 电路图

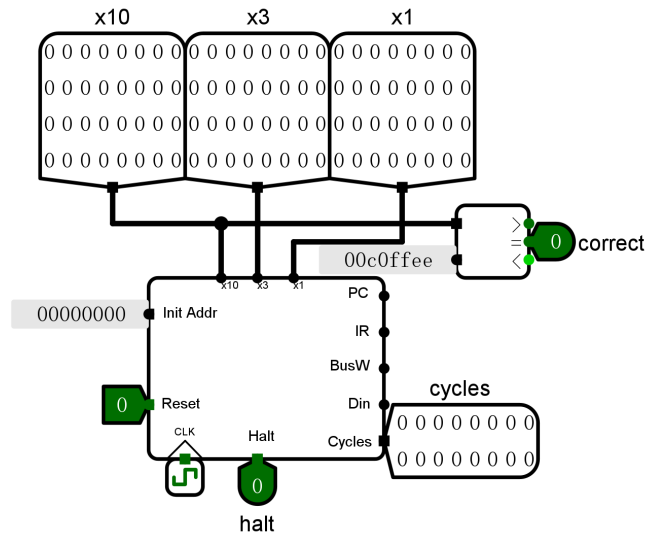


图 17: 测试电路图

随后撰写一个测试脚本 `autotest.sh`，使用 Logisim 的命令行工具进行自动化测试。

如果是 windows 系统，可以使用 PowerShell 运行 `autotest.ps1`，（下载后将文件后缀名改为 .ps1）同样使用 Logisim 的命令行工具进行自动化测试。

5.2 测试结果

指令	x10	x3	x1	时钟周期
add	0x00C0FFEE	0x00000026	0x00000010	0x01C9
addi	0x00C0FFEE	0x00000019	0x00000021	0x00EA
and	0x00C0FFEE	0x0000001B	0x11111111	0x01DD
andi	0x00C0FFEE	0x0000000E	0x00FF00FF	0x00BE
auipc	0x00C0FFEE	0x00000003	0x00000000	0x0033
beq	0x00C0FFEE	0x00000015	0x00000003	0x011B
bge	0x00C0FFEE	0x00000018	0x00000003	0x012D
bgeu	0x00C0FFEE	0x00000018	0x00000003	0x0146
blt	0x00C0FFEE	0x00000015	0x00000003	0x011B
bltu	0x00C0FFEE	0x00000015	0x00000003	0x0134
bne	0x00C0FFEE	0x00000015	0x00000003	0x011B
jal	0x00C0FFEE	0x00000003	0x00000003	0x002F
jalr	0x00C0FFEE	0x00000007	0x00000000	0x006B
lb	0x00C0FFEE	0x00000013	0x00002000	0x00ED
lbu	0x00C0FFEE	0x00000013	0x00002000	0x00ED
lh	0x00C0FFEE	0x00000013	0x00002000	0x00F9
lhu	0x00C0FFEE	0x00000013	0x00002000	0x0100
lui	0x00C0FFEE	0x00000006	0xFFFFF800	0x0039
lw	0x00C0FFEE	0x00000013	0x00002000	0x0103
or	0x00C0FFEE	0x0000001B	0x11111111	0x01E0
ori	0x00C0FFEE	0x0000000E	0x00FF00FF	0x00C5
sb	0x00C0FFEE	0x00000017	0x00000001	0x01A6
sh	0x00C0FFEE	0x00000017	0x00003001	0x01DB
sll	0x00C0FFEE	0x0000002B	0x00000400	0x01E5
slli	0x00C0FFEE	0x00000019	0x00000021	0x00E9
slt	0x00C0FFEE	0x00000026	0x00000010	0x01C3
slti	0x00C0FFEE	0x00000019	0x00FF00FF	0x00E5
sltiu	0x00C0FFEE	0x00000019	0x00FF00FF	0x00E5
sltu	0x00C0FFEE	0x00000026	0x00000010	0x01C3

指令	x10	x3	x1	时钟周期
sra	0x00C0FFEE	0x0000002B	0x00000400	0x01F8
srai	0x00C0FFEE	0x00000019	0x00000021	0x01F8
srl	0x00C0FFEE	0x0000002B	0x00000400	0x01F2
srli	0x00C0FFEE	0x00000019	0x00000021	0x01F2
sub	0x00C0FFEE	0x00000025	0x00000010	0x01C1
sw	0x00C0FFEE	0x00000017	0x12233001	0x01E2
xor	0x00C0FFEE	0x0000001B	0x11111111	0x01DF
xori	0x00C0FFEE	0x0000000E	0x00FF00FF	0x00C7

6 计算机系统基础 PA 程序测试实验

使用编译好的文件进行测试。

7 思考题

7.1 如何在单 CPU 上实现多任务处理

要同时处理多个任务，我们需要对这几个任务分配编号、状态存储和内存空间，使得它们在时间和空间上相互隔离。对于每个任务，我们需要保存它的程序计数器、寄存器状态和内存状态，以便在切换任务时能够恢复它的执行状态。我们可以使用一个调度器来管理这些任务，按照一定的策略来决定哪个任务获得 CPU 的使用权。当一个任务被切换出去时，调度器会保存它的状态，并加载下一个任务的状态，使得它能够继续执行。

7.2 如何实现键盘输入、TTY 输出部件等输入输出设备的数据访问，构建完整的计算机系统

我们可以通过内存映射 I/O 的方式来实现输入输出设备的数据访问。对于每个输入输出设备，我们可以为它分配一个特定的内存地址，当 CPU 访问这个地址时，实际上是在与这个设备进行通信。例如，对于键盘输入，我们可以将键盘的输入数据映射到一个特定的内存地址，当 CPU 读取这个地址时，就能够获取到键盘输入的数据。对于 TTY 输出，我们可以将 TTY 的输出数据映射到另一个特定的内存地址，当 CPU 写入这个地址时，就能够将数据发送到 TTY 进行显示。通过这种方式，我们就能够实现输入输出设备的数据访问，从而构建一个完整的计算机系统。

7.3 如何在单周期 CPU 基础上实现多周期 CPU

要在单周期 CPU 的基础上实现多周期 CPU，我们需要将指令的执行过程分解成多个阶段，每个阶段对应一个时钟周期。通常情况下，我们可以将指令的执行过程分为取指、译码/读寄存器、执行、访存和写回等几个阶段。在每个阶段，我们只执行指令的一部分操作，这样就能够多个时钟周期内完成一条指令的执行。为了实现多周期 CPU，我们还需要增加一些控制信号来控制每个阶段的操作，并且需要设计一个状态机来管理这些阶段的切换。通过这种方式，我们就能够在单周期 CPU 的基础上实现多周期 CPU，从而提高 CPU 的性能和效率。

7.4 如何在单周期 CPU 基础上实现流水线 CPU

要在单周期 CPU 基础上实现流水线 CPU，核心是在数据通路中按功能切分出取指、译码、执行、访存和写回等多个级段，并在各级之间强行插入“流水线寄存器”来暂存数据和随行传递的控制信号，从而让多条指令能够在同一时刻处于不同阶段重叠执行；同时，必须额外引入数据转发单元和流水线冲刷控制逻辑，以解决因指令重叠导致的数据依赖和分支跳转冲突，最终在不改变指令延迟的前提下，极大地提高 CPU 的整体吞吐率和时钟频率。

8 RV32I 测试表

fib1.asm		指令存储器加载: fib1.hex		
M[0x2000] 加载	0x14	0x20	0x32	0x40
执行周期数	130	202	310	394
M[0x2004] 结果	0x1a6d	0x213d05	0xee333961	0x61ca20bb

mul1.asm		指令存储器加载: mul1.hex	
M[0x2000], M[0x2004] 加载	0x7b, 0x1c8	0xff00, 0xff00	
执行周期数	67	113	
R[7] 结果	0xdb18	0xfe010000	

div1.asm		指令存储器加载:div1.hex	
M[0x2000],M[0x2004] 加载	0x64,0x7	0xff00,0xfe	
执行周期数	64	1037	
R[7],R[5] 结果	0xe,0x2	0xfe,0x2	
array-add.asm		指令存储器加载:array-add.hex	
		数据存储器 RAM0 加载:array-add.dat	
执行周期数		87	
M[0x200] 及其后共 10 个数据		0xfffffee,0xfffffd8,0xfffffa1,0xfffffd0,0xb3, 0xb6,0xfffff40,0x65,0x89,0xfffffa2	